DashVMC

Real-Time Discrete World Model Control in Geometry Dash

Contributors
Florent Tariolle[1], Florian Yger[1,2]

[1]**INSA Rouen Normandy** [2]**LITIS**

World-model agents are usually evaluated in simulators that can wait for the policy; live games impose the opposite constraint, requiring capture, prediction, and action before the next frame. We present *DashVMC*, which learns a compact, action-conditioned world model from approximately two hours of recorded Geometry Dash gameplay. To test whether the learned dynamics are actionable, a controller is initialized by behavioural cloning (BC) and refined with Proximal Policy Optimization (PPO) entirely in frozen-model rollouts, without further interaction with the live game. Across three controller seeds, the refined policies improve mean live survival over their BC initializations on all three official levels and preserve the same direction on a held-out community layout. At deployment, the controller reuses the world model's temporal representation while skipping visual generation, enabling 60-FPS control on a consumer GPU. Action-conditioned continuations and rollout diagnostics show that the model remains useful for control despite imperfect long-horizon fidelity.

Project page: tariolle.github.io/dash-vmc
Code: github.com/Tariolle/dash-vmc
Correspondence: florent.tariolle@insa-rouen.fr

1 Introduction

A world model learns an environment's transition dynamics, conventionally written $p_\theta(s_{t+1} \mid s_t, a_t)$: given a state s_t and action a_t, it predicts the state that follows (LeCun, 2022). When privileged state is unavailable, it can learn this relation directly from recent observations and actions, then roll its predictions forward so that an agent can practice without further interaction with the real environment (Ha and Schmidhuber, 2018). Discrete approaches such as IRIS encode images into tokens and model their evolution with an autoregressive transformer (Micheli et al., 2023), whereas DIAMOND uses diffusion to preserve fine visual detail (Alonso et al., 2024). DashVMC asks whether a world model can serve two roles at once: provide an environment in which a policy can improve, and provide temporal understanding quickly enough to control a live game. It combines parallel next-grid prediction for inexpensive imagined rollouts with a decoder-free context path for live control. Relative to IRIS-style autoregressive token generation, DashVMC predicts the next 64-token grid in parallel and reuses only the transformer's temporal state at deployment.

Most world-model agents are evaluated in synchronous environments that wait for the policy. A live, non-paused application reverses that contract: screen capture, perception, inference, and action dispatch must finish before the action becomes stale.

Geometry Dash is a small but unforgiving test of this contract. The avatar moves continuously, the action space is binary, and a mistimed jump causes immediate death. Levels are deterministic, but the agent observes only captured pixels and acts through keyboard events; it does not receive emulator state during control. The apparent simplicity therefore isolates a demanding question: can an agent learn when to act from pixels, improve without touching the live game, and still react within narrow timing windows on modest hardware?

DashVMC follows the perception–dynamics–control organization of World Models (Ha and Schmidhuber, 2018). It compresses each frame into a small discrete grid, learns how that grid changes under jump or idle

actions, and trains a lightweight action model in the resulting imagined environment. Behavioural cloning (BC) first teaches the action model the broad rhythm of human play; PPO (Schulman et al., 2017) then refines it through repeated imagined consequences. When the trained policy returns to the live game, it reuses the world model's temporal state while omitting next-grid prediction and decoding.

Our contributions are:

1. a compact discrete, action-conditioned world model learned from approximately two hours of offline gameplay, whose rollouts support policy refinement without further live-game interaction;
2. a shared temporal representation used both to generate action-conditioned continuations and to support a decoder-free control path sustaining 60 FPS on a consumer GPU; and
3. evidence that the learned dynamics are actionable: controllers refined only in frozen-model rollouts improve over their exact BC parents on all three official levels across three seeds, with the same direction on a held-out layout.

We evaluate these claims through three retained BC-to-PPO controller pairs, one held-out community layout, a matched-action intervention, end-to-end latency measurements, and open-ended rollouts.

2 Related work

World models and imagination. The original Vision–Memory–Controller pipeline compressed observations with a VAE, modeled latent dynamics with an MDN–RNN, and demonstrated in VizDoom that a compact controller could be optimized entirely in learned dreams and transferred to the real environment (Ha and Schmidhuber, 2018). Subsequent imagination-based agents explore different representations and dynamics models: IRIS, the closest architectural reference for DashVMC, combines a discrete autoencoder with an autoregressive transformer for data-efficient Atari learning (Micheli et al., 2023); Δ-IRIS separates stochastic changes into discrete tokens from continuous context tokens to reduce sequence cost (Micheli et al., 2024); and DIAMOND uses diffusion without discrete compression for imagination-trained control and also demonstrates an interactive Counter-Strike neural game engine (Alonso et al., 2024). DreamerV3 demonstrates broad control with categorical recurrent state-space latents (Hafner et al., 2025), while TWISTER augments transformer world-model training with action-conditioned contrastive predictive coding to learn temporal representations over longer horizons (Burchi and Timofte, 2025). At larger generative scales, related action-conditioned simulators include MIRA, which models tightly coupled four-player Rocket League dynamics with latent diffusion (Hu et al., 2026); MineWorld, which autoregressively models interleaved discrete visual and action tokens with parallel spatial decoding (Guo et al., 2025); and Matrix-Game 2.0, which uses few-step diffusion for streaming generation (He et al., 2025). Within Geometry Dash specifically, earlier screenshot-based control used direct DQN and imitation learning under a DLL-fabricated timestep, and highlighted that narrow jump windows can make nearly identical frames require different actions (Li and Rafferty, 2017). Our protocol leaves the game clock running and supplies the controller with temporal context through the learned world model.

FSQ tokenization and policy initialization. FSQ replaces learned vector-quantization codebooks with bounded scalar levels, avoiding commitment losses and codebook collapse while exposing an integer coordinate lattice (Mentzer et al., 2024); we use the improved iFSQ bounding function (Lin et al., 2026). For policy initialization, demonstration-based RL has repeatedly shown that supervised behaviour can reduce the expensive early interaction phase (Hester et al., 2018; Baker et al., 2022). Therefore we use the demonstrations only to initialize the controller: the final policy is selected after PPO improvement in the learned environment.

Representation anchors and alternatives. Joint-embedding prediction can avoid spending capacity on high-magnitude input detail that is irrelevant or unpredictable from context (Van Assel et al., 2025), and LeWorldModel demonstrates stable end-to-end predictive learning from pixels with continuous latents (Maes et al., 2026). DashVMC instead applies deterministic preprocessing that converts each cropped RGB capture

to grayscale and then to a Sobel edge map, removing much appearance detail before learning and making reconstruction a comparatively inexpensive anchor for the shared discrete token space. Among discrete tokenizers, Gaussian Quant reports stronger rate–distortion performance than FSQ on large image benchmarks (Xu et al., 2025). Whether those gains transfer to low-resolution Sobel edge maps and downstream world-model control remains untested.

3 DashVMC

DashVMC is organized around a simple loop: recorded play trains a world model, the action model is refined in rollouts from the frozen dynamics model, and the resulting controller is returned to the live game. The visual encoder, dynamics model, and action model are trained in sequence and then frozen for deployment. Figure 1 shows how the same learned state supports both imagined practice and live reaction.

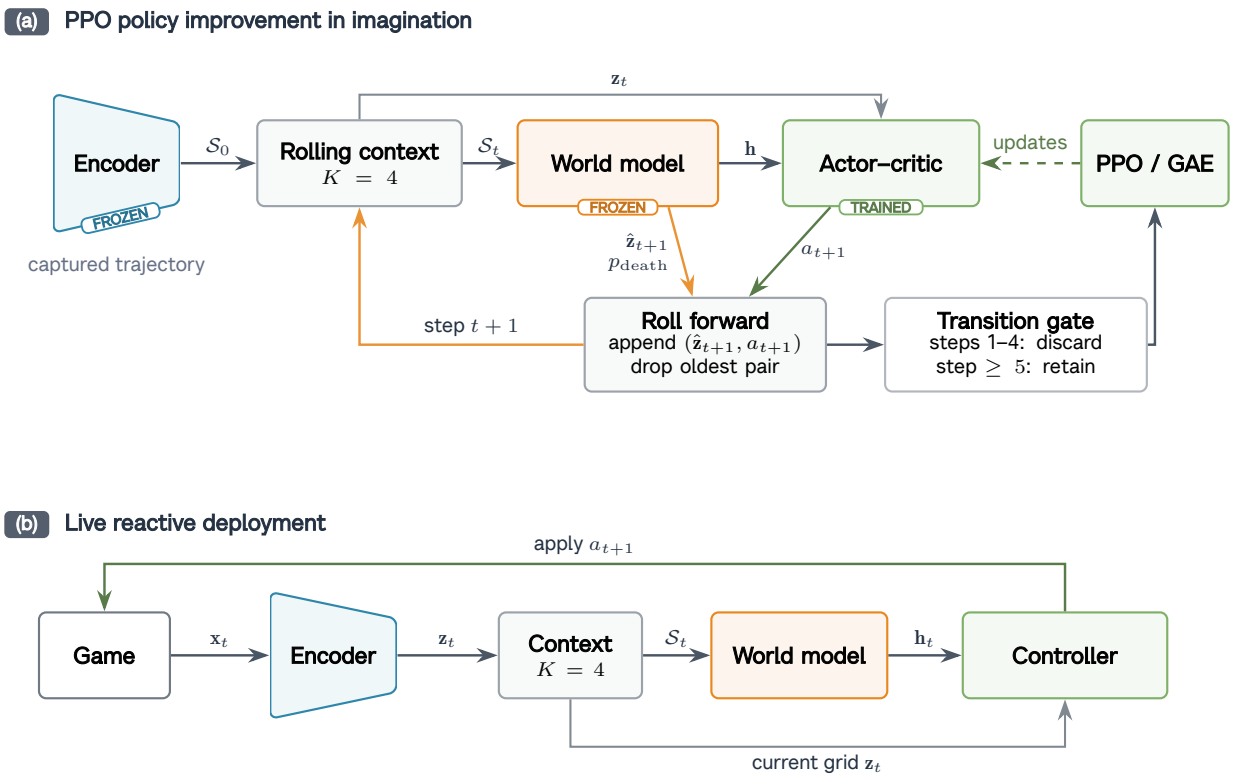

Figure 1 Learning in imagination and acting in the live game. **(a)** A recorded context seeds the frozen world model; the action model chooses what to do, the world model predicts the consequence, and PPO updates only the action model. **(b)** In the live game, the encoder and temporal context provide the state used for action selection, while visual generation is skipped.

3.1 Visual tokenization

Each RGB capture is cropped, converted to grayscale, filtered with Sobel edges, and resized to 64×64. A convolutional autoencoder maps the result through three stride-2 stages to a four-channel 8×8 latent map. Each spatial vector is independently quantized with iFSQ levels $[8, 5, 5, 5]$, using the bounding function $2\,\mathrm{sigmoid}(1.6z) - 1$, and yields 64 token IDs from 1000 implicit codes (Mentzer et al., 2024; Lin et al., 2026). The tokenizer is trained on adjacent frames with reconstruction, temporal-slowness, and latent-uniformity objectives (Xia et al., 2025). Once selected, its encoder is frozen; its decoder is used to visualize dreams but is not required by the action model.

3.2 Action-conditioned dynamics

The dynamics model is an eight-layer, width-384 transformer that receives four recent frame–action pairs. Each frame block contains 64 visual tokens and one ALIVE/DEATH status token, with action tokens inserted between blocks; a final masked block requests the next frame. Attention is bidirectional within a frame and causal across frame/action blocks, so all next-frame positions are predicted in one forward pass without seeing their targets. An explicit action token separates consecutive frames, so the model can answer the question central to policy learning: what changes if the action model jumps rather than idles? It predicts all 64 codes of the next grid in parallel, together with an ALIVE/DEATH score, and exposes a temporal summary $\mathbf{h}_t$, taken from the most recent action-token position, to the action model. Parallel prediction keeps imagined steps inexpensive, while the rolling context lets generated consequences become the starting point for the next decision.

Training combines next-grid prediction with action-conditional contrastive predictive coding of weight 0.1 (Burchi and Timofte, 2025; van den Oord et al., 2018), context corruption, and an auxiliary death objective. For context corruption, 5% of context tokens are replaced uniformly and another 5% with adjacent FSQ codes. The token loss uses focal modulation (Lin et al., 2017) and structured label smoothing (SLS), which assigns its smoothing mass to nearby coordinates in the FSQ lattice rather than uniformly across unrelated codes. For target code i with lattice coordinate $\mathbf{c}_i$,

$$q(j \mid i) = \begin{cases} 1 - \varepsilon, & j = i, \\ \varepsilon \dfrac{\exp(-\|\mathbf{c}_i - \mathbf{c}_j\|_2^2 / 2\sigma^2)}{\sum_{\ell \neq i} \exp(-\|\mathbf{c}_i - \mathbf{c}_\ell\|_2^2 / 2\sigma^2)}, & j \neq i, \end{cases} \tag{1}$$

with $\varepsilon = 0.1$ and $\sigma = 0.9$. Writing $H(q, p) = -\sum_j q(j \mid i) \log p_j$, focal modulation is applied once to the aggregate soft-target loss as $(1 - e^{-H})^2 H$, and the output projection is tied to the token embedding. This local target reflects the observation that neighbouring FSQ codes decode to visually similar patches; it is a design choice for this representation rather than a general claim about world-model training.

3.3 Controller and latent policy improvement

The 45,546-parameter actor–critic embeds the current token grid, processes it with two convolutions and max pooling, and concatenates the resulting 256-dimensional feature with the dynamics model's temporal summary $\mathbf{h}_t$. Separate heads predict jump probability, value, and an eight-action auxiliary output covering the current and next seven actions. Its training has two stages with different purposes. BC teaches it to reproduce the broad timing of expert actions; PPO then lets it depart from imitation and discover, through repeated imagined consequences, actions that survive longer.

Each PPO iteration generates 512 imagined episodes of at most 45 steps from recorded four-frame contexts, with greedy next-grid predictions and sampled Bernoulli actions. The first four generated transitions refresh the context but are excluded from PPO because the recorded approach may already make a collision unavoidable. The remaining horizon covers the obstacles visible in the seed while avoiding long updates dominated by geometry invented beyond it. Rollouts terminate when the death logit exceeds the ALIVE logit; because death frames are oversampled 5× during dynamics training, this score is used as an operational boundary rather than a calibrated probability. Reward is +1 per survived step and −0.25 per jump, preventing an always-jump local optimum. We use Generalized Advantage Estimation (GAE) (Schulman et al., 2015) with $\gamma = 0.995$, $\lambda = 0.95$, PPO clip 0.2, and auxiliary-action loss weight 0.1.

3.4 Reactive deployment path

Training asks the world model to generate consequences; live control only asks it to interpret what has just happened. Desktop Duplication captures only the 1032×1032 model crop; CPU grayscale conversion followed by CUDA Sobel filtering and area resizing produces the 64×64 observation. Each frame is encoded and appended to a GPU-resident context, and the transformer runs context prefill only to produce $\mathbf{h}_t$. The next-grid predictor and image decoder are skipped, reducing the deployed path to 15.57 million parameters; the encoder uses `torch.compile`, and transformer context encoding uses a CUDA Graph after warm-up.

The accelerated preprocessing path was checked against the recorded-data pipeline on synthetic and fresh live captures and produced byte-identical observations. This is a reactive controller rather than an online planner: the policy has already performed its search through experience during offline imagined training.

4 Experimental setup

Data and training. Before augmentation, the corpus contains 4,264 action-labelled gameplay episodes, or 251,963 frames and approximately 2.33 hours at the nominal 30-FPS capture cadence. Each captured frame is paired with the current space-bar state (IDLE/JUMP); process memory marks death/respawn boundaries for segmentation and scoring but never enters the controller. It includes deliberate failures, official Levels 1–7, several community levels, and 36 expert trajectories or segments totaling less than 20 minutes. Mechanics outside the selected scope, including gravity inversion, are excluded. Failure and expert trajectories are each split 90/10 with seed 42 at the base-episode level. During tokenizer training, each adjacent-frame pair receives a shared two-dimensional translation whose horizontal and vertical offsets are sampled independently and uniformly from $[-4, 4]$ pixels. For dynamics training, the frozen encoder instead tokenizes each base episode at vertical offsets $\{-4, -2, 0, 2, 4\}$, expanding the corpus to 21,320 episode variants. All temporal windows and translated variants inherit the assignment of their base episode; a SHA-256 audit of the 4,264 usable base frame–action trajectory pairs found no exact duplicates before augmentation. The tokenizer, dynamics model, and BC controller use this split, whereas PPO draws rollout seeds from the full recorded corpus and uses 512 fixed contexts from the 10% stratum for checkpoint selection. Tokenizer and dynamics training use seed 42, while controller training is repeated with seeds 43–45 under the same frozen representation and world model. The first two stages run sequentially in BF16 on one A100, controller training runs in BF16 on one H200 at a time, and live inference and local latency evaluation use the same RTX 2060 with PyTorch 2.11.0+cu126 (CUDA 12.6). Table 1 records the optimization and selection details that materially determine the frozen system.

Table 1 Training and checkpoint selection. Tokenizer/dynamics learning rates use cosine decay after any stated warm-up.

Stage	Optimization	Selection
Tokenizer	Adam; 1,000 epochs; batch 2,048; cosine LR from 10^{-3} to 10^{-5}; slowness 0.1; uniformity 0.01; validation every 10 epochs.	Min. loss (epoch 920).
Dynamics	AdamW; 200 epochs × 500 steps; batch 512; LR $2\times10^{-3}\rightarrow5\times10^{-5}$ after 5-epoch warm-up; weight decay 0.01; dropout 0.1; max grad. norm 1.0; death frames oversampled 5×.	Max. death F1 (epoch 139).
BC	Three seeds; AdamW; 50 epochs/seed; batch 512; LR 10^{-3}; weight decay 10^{-4}; positive jump weight 1.5.	Min. loss (10; 10; 11).
PPO	Three seeds; Adam; 15,000 iterations/seed; 512 rollouts/iteration; four update epochs; minibatch 512; LR 10^{-4}; entropy/critic weights 0.01/0.5; ratio clip 0.2; max grad. norm 0.5.	Best dev. survival (12,420; 5,090; 12,280).

The historical tokenizer and dynamics stages total approximately 12.6 A100 hours; the three controller replications add 51.54 H200 hours and 45,000 PPO iterations.

Checkpoint and live evaluation. PPO checkpoint selection uses survival on 512 fixed latent development contexts every ten iterations. All live checkpoints are then frozen and act deterministically from pixels alone; Auto-Retry standardizes restarts but provides no policy input. For each seed, we compare PPO with the exact BC checkpoint that initialized it over 25 scored attempts on *Stereo Madness*, *Back on Track*, and *Polargeist*. This pairing is by controller lineage: the 25 BC and 25 PPO live attempts are separate samples rather than attempt-matched trials. The same protocol is used on two auxiliary layouts. *Stereo Madness Copy* begins at the official level's first ship section, revealing later control behaviour that the official Level 1–3 runs do not reach. *Stereo INSANE Nerfed* is a community layout first accessed after the corpus and checkpoints were frozen, providing a held-out test within the supported mechanics. To validate the earlier 30-FPS evaluations, a cadence check compares the identical frozen PPO checkpoint on Stereo Madness Copy over 25 optimized-path attempts at 60 FPS and 20 earlier diagnostic-path attempts at 30 FPS.

The primary endpoint is frames survived; the cross-cadence probe uses episode wall time because frame counts change with cadence. Within-policy variation is the sample standard deviation over attempts. PPO-minus-parent-BC intervals independently resample the 25 attempts from each frozen policy 50,000 times. These attempt-level intervals characterize deployment variability rather than controller-training uncertainty; the three controller-seed differences provide the replication evidence. For the official-level aggregate, each level is normalized from its fixed historical no-op mean to the best observed retained-policy seed mean. We report the mean, interquartile mean (IQM), and empirical-reference gap with 50,000 task-stratified bootstrap resamples of controller seeds following Agarwal et al. (2021). These seed-level summaries complement, rather than pool, the three controller replications.

Latency and continuation protocols. Live latency is measured from capture through action dispatch over 5,000 consecutive frames of the optimized 60-FPS path. For the qualitative intervention, the world model is initialized from a fixed encoded four-frame prefix immediately before a spike. Greedy continuations differ only in the first conditioned action, jump or idle; all subsequent actions are idle. Post-hoc quantitative diagnostics re-encode the unaugmented base trajectories in the fixed validation stratum with the selected tokenizer. They contain 22,745 next-frame windows from 413 usable trajectories; twelve additional failure trajectories are shorter than the five frames required for one context–target window. For the paired action intervention, we hold each recorded four-frame context fixed and flip only the final binary action, which directly precedes the target frame. The reported interval resamples whole trajectories 5,000 times, stratified by failure or expert source. For autoregressive fidelity, predicted grids are fed back greedily while replaying the recorded future actions. The standard cohort contains 1,024 starts, capped at 64 per trajectory, from 150 failure and three expert trajectories; an exploratory 200-step cohort contains 256 starts from the only four sufficiently long validation trajectories.

External transition-latency protocol. We also time the native imagined transition of DashVMC and released Pong checkpoints of IRIS and DIAMOND on the same RTX 2060. Each model runs in a fresh process at batch size one with FP32 eager execution; 30 warm-up transitions are followed by 100 synchronized transitions, repeated twice. DashVMC predicts a latent grid and status, IRIS generates and decodes 16 tokens plus reward and termination, and DIAMOND performs three diffusion steps plus reward and termination. Because these native interfaces produce different representations, the comparison concerns transition cost rather than generation quality or control.

5 Results

5.1 Policy refinement in the live game

The central question is whether policy improvement learned only through frozen-model rollouts transfers beyond imitation to the live game. The selected tokenizer reconstructs held-out edge maps at 34.10 dB PSNR. On the unaugmented validation trajectories, the dynamics model reaches 29.58% visual-token accuracy and 0.793 death F1. Because validation death F1 selected the dynamics checkpoint, the latter is a development-stratum estimate rather than an untouched test result. These component metrics establish a usable but imperfect imagined environment; the live comparison below tests whether it nevertheless teaches better actions.

Table 2 Live survival on all five layouts. No-op entries are mean frames ± standard deviation over 10 attempts and are shared across seeds. BC/PPO entries use 25 attempts; Δ is PPO minus its exact parent BC [95% attempt-level bootstrap interval]. The first three are official levels, Stereo Madness Copy begins in ship form, and Stereo INSANE Nerfed is held out.

Level	Policy	Seed 43	Seed 44	Seed 45
Stereo Madness	No-op		shared: 46.2 ± 0.7	
	BC	120.2 ± 91.8	130.6 ± 76.0	156.0 ± 100.2
	PPO	$\mathbf{280.1 \pm 23.3}$	$\mathbf{280.4 \pm 33.5}$	$\mathbf{308.8 \pm 68.6}$
	Δ [95% CI]	159.9 [122.7, 195.4]	149.8 [116.8, 180.6]	152.8 [106.7, 199.8]
Back on Track	No-op		shared: 63.4 ± 0.7	
	BC	123.5 ± 81.5	111.1 ± 60.3	120.2 ± 69.6
	PPO	$\mathbf{260.0 \pm 55.5}$	$\mathbf{195.9 \pm 126.5}$	$\mathbf{209.4 \pm 130.0}$
	Δ [95% CI]	136.5 [98.8, 174.4]	84.8 [31.9, 139.6]	89.2 [34.6, 147.2]
Polargeist	No-op		shared: 41.5 ± 0.7	
	BC	52.4 ± 16.7	49.0 ± 16.5	46.2 ± 9.2
	PPO	$\mathbf{65.2 \pm 33.4}$	$\mathbf{68.3 \pm 44.1}$	$\mathbf{63.1 \pm 32.9}$
	Δ [95% CI]	12.8 [−0.2, 28.6]	19.4 [3.0, 39.0]	16.8 [5.7, 31.5]
Stereo Madness Copy	No-op		shared: 135.0 ± 0.0	
	BC	366.8 ± 172.8	285.7 ± 153.5	299.4 ± 175.2
	PPO	$\mathbf{435.6 \pm 196.1}$	$\mathbf{495.8 \pm 137.5}$	$\mathbf{546.8 \pm 94.1}$
	Δ [95% CI]	68.8 [−33.3, 166.1]	210.1 [127.9, 285.5]	247.4 [167.6, 320.7]
Stereo INSANE Nerfed	No-op		shared: 46.1 ± 0.8	
	BC	104.8 ± 72.1	102.1 ± 64.0	97.8 ± 57.9
	PPO	$\mathbf{290.4 \pm 48.6}$	$\mathbf{342.8 \pm 80.8}$	$\mathbf{266.7 \pm 128.9}$
	Δ [95% CI]	185.5 [151.1, 217.6]	240.6 [200.8, 280.2]	168.8 [115.4, 223.9]

The shared no-op rows of Table 2 establish the level-dependent difficulty floor: BC exceeds no-op on every layout, and PPO then improves over its parent imitation policy in all fifteen seed-level comparisons. Across the three controller seeds, the paired improvements are 154.2 ± 5.2 frames on *Stereo Madness*, 103.5 ± 28.7 on *Back on Track*, and 16.3 ± 3.3 on *Polargeist*, where the variation is the sample standard deviation across the three seed differences. Table 3 summarizes the same official-level results from seed-level policy means rather than attempt-level samples. PPO raises both aggregate location estimates and reduces the empirical-reference gap. Every attempt-level interval on the first two levels excludes zero. The smaller Polargeist improvement is positive for all three seeds, although seed 43's deployment-variability interval marginally includes zero; seeds 44 and 45 exclude it. Polargeist introduces a new timing demand—yellow orbs that require a second input while airborne—which helps explain why improvement is smaller there.

Table 3 Aggregate normalized survival over official levels. Scores map no-op to 0 and the best observed retained-policy seed mean to 1. Entries are estimates [95% seed-level stratified-bootstrap intervals]; higher is better except for the empirical-reference gap.

Metric	BC	PPO
Mean ↑	0.302 [0.261, 0.346]	**0.877** [0.818, 0.942]
IQM ↑	0.295 [0.264, 0.350]	**0.894** [0.826, 0.978]
Empirical-reference gap ↓	0.698 [0.654, 0.739]	**0.123** [0.058, 0.182]

The final two level blocks show that the same no-op–BC–PPO progression extends beyond the three official layouts. On Stereo Madness Copy, seed 43's attempt-level interval includes zero while seeds 44 and 45 exclude it; the mean paired improvement across seeds is 175.4 ± 94.2 frames. On Stereo INSANE Nerfed, all three intervals exclude zero and the mean paired improvement is 198.3 ± 37.6 frames. Stereo INSANE Nerfed shows that the refinement transfers to an unseen layout within the supported mechanics. Stereo Madness Copy is not held out, but it exposes a different qualitative ability: starting at the official level's first ship section, the agent maintains altitude and avoids obstacles with strong ship control. The 60-FPS cadence probe on Stereo Madness Copy yields similar observed real-time survival: mean recorded episode wall time is 17.72 s at 60 FPS versus 17.79 s at 30 FPS, with a 95% bootstrap interval of $[-1.04, 0.89]$ s for the mean difference (60 FPS minus 30 FPS). On this checkpoint and layout, the similarity supports treating earlier 30-FPS evaluations as representative after 60-FPS acceleration.

5.2 Live control at game speed

Reactive play requires the complete perception-to-action loop, not just policy inference, to finish within each 16.67-ms frame. At 60 Hz, the controller can revise the binary key state every frame—60 decisions per second, not 60 distinct key presses. Over 5,000 frames, the complete capture-to-action path averages 12 ms while the same RTX 2060 also renders the game. The mean leaves 4.65 ms (27.9%) of the frame period unused, equivalent to 83.2 unpaced loops per second. Mean and median differ by only 0.065 ms, and 11/5,000 frames (0.22%) exceed the period, indicating stable throughput across capture, preprocessing, model inference, and key-state update. This doubles the temporal resolution of the 30-FPS demonstrations. Qualitatively, the ship policy uses sustained thrust and release intervals: ship motion remains correctable in flight, whereas a cube jump is largely ballistic until landing and therefore demands sparser, more precise timing. The smooth, low-duty-cycle behaviour is consistent with PPO's per-step jump penalty, although it does not isolate that penalty's effect.

5.3 Cost of an imagined step

Table 4 compares the cost of one native imagined transition. DashVMC's native imagined transition averages 11.05 ms, compared with 49.30 ms for DIAMOND and 177.83 ms for IRIS, corresponding to 4.5× and 16.1× lower latency, respectively. Only DashVMC fits the 16.67-ms 60-FPS period on this hardware.

Table 4 Batch-1 native imagined-transition latency on an RTX 2060 (synchronized run means in milliseconds). Interfaces differ, so this compares operational cost rather than quality.

Model	Run 1	Run 2	Mean
DashVMC	11.04	11.06	**11.05**
DIAMOND	50.13	48.46	49.30
IRIS	176.94	178.73	177.83

The short path follows from predicting all 64 target tokens in parallel after a context prefill and keeping PPO rollouts in latent space: the seed is encoded once, predicted grids are fed back directly, and no decoder reconstructs pixels. A fixed four-frame context and batched rollouts further reduce the repeated cost. Because every imagined step invokes the dynamics model, these savings compound through PPO and make single-GPU experiments more affordable. This benchmark concerns generation, whereas live control skips next-grid prediction and decoding entirely. The interfaces also differ, so it explains operational cost without establishing generation-quality superiority or whether retrained IRIS or DIAMOND policies could control this game.

5.4 Action sensitivity and autoregressive fidelity

The learned dynamics model can be rolled out beyond the 45-step training horizon by feeding each predicted grid and newly selected action back into its context. In interactive rollouts it reproduces recognizable Geometry Dash structure, including scrolling terrain, camera movements, height changes, collisions, and avatar transformations. Some continuations remain coherent for long stretches, while others eventually repeat motifs, empty the scene, or produce impossible geometry. Separate free-running demonstrations include continuations lasting hundreds of generated steps. Video demonstrations of interactive level continuations are available on the project page.

Figure 2 tests action sensitivity while holding the visual context fixed. The two futures begin from the same recorded moment, yet a single changed action produces the expected qualitative divergence: the jump branch clears the spike and lands, while the idle branch collides and terminates. This deliberately short matched-context diagnostic isolates action conditioning; it is not intended to measure long-horizon coherence. The corresponding corpus-wide intervention supports the same conclusion. Across all 22,745 validation transitions, the factual action has lower next-frame negative log-likelihood than its flipped counterpart in 69.25% of contexts. The trajectory-mean advantage is 0.165 nats per visual token (95% episode-bootstrap interval [0.150, 0.181]); factual-action token accuracy is 29.58% versus 27.77% after flipping, and 11.03% of argmax token predictions change.

Figure 2 Action-conditioning diagnostic from one shared four-frame context. The rows are aligned at $t+1$ and $t+5$: changing only the first future action makes the idle branch collide and terminate, while the jump branch clears the spike and lands by $t + 13$.

Table 5 Recorded-action rollout diagnostics. Accuracy and decoder-space PSNR compare predicted with recorded grids; JS compares pooled token marginals. Standard: 1,024 starts from 153 trajectories; extended: 256 starts from four long trajectories.

Cohort	Horizon	Token acc. ↑	PSNR ↑	Token JS ↓
Standard	1	30.12%	32.82	0.0037
	5	19.23%	28.24	0.0044
	10	14.04%	25.21	0.0050
	20	8.39%	22.48	0.0078
	45	2.86%	20.00	0.0205
Extended	45	2.91%	19.84	0.0412
	100	1.49%	19.62	0.0498
	200	0.76%	19.57	0.0880

Table 5 separates visual structure from exact recorded-trajectory fidelity. Rapid decay is partly expected in this left-scrolling game: every step reveals previously off-screen geometry at the right edge, so after many steps much of the target frame was absent from the four-frame seed. Exact token accuracy therefore conflates accumulated dynamics error with prediction of newly revealed content; the 2.86% at 45 steps is not a pure measure of compounding error. Pooled token marginals drift more slowly, but this weak distributional check and the four-trajectory extended cohort cannot establish perceptual realism. We therefore treat the hundred-step examples as qualitative evidence that generation can remain structured, not as evidence of faithful long-horizon simulation.

Development observations. Uncontrolled development runs suggested that BC saved about 2,000 PPO iterations, width 512 improved training metrics but not live play, and an $\mathbf{h}_t$-only MLP trained 2–5× faster but progressed less in the game; we therefore retained BC, width 384, and the token-grid input.

6 Limitations

DashVMC is a single-game study with binary actions and deterministic layouts. Broader generalization therefore remains open, and the held-out test covers one community layout rather than new mechanics. The three controller seeds share one frozen tokenizer and dynamics model, so they measure controller-training variability rather than end-to-end model uncertainty. Training was limited to one GPU at a time, constraining the breadth of the experimental matrix. We therefore prioritized end-to-end validation and targeted component ablations over multi-seed ablations and model- or data-scaling curves. Recorded-future fidelity falls rapidly in long dreams, and geometry can drift or repeat, pointing to the limits of the modest training corpus. The 200-step quantitative cohort is drawn from only four sufficiently long validation trajectories, while token-marginal similarity is not a perceptual-quality metric. Live evaluation through the original desktop application proceeds at wall-clock speed and is difficult to accelerate or parallelize, limiting the scale of deployment tests. Finally, the reported speed reflects the tested hardware and compact edge representation; richer games may require a different balance between visual detail and reaction time.

7 Conclusion

DashVMC shows that a world model need not reproduce the recorded future exactly to improve a policy. Despite long-rollout drift, policies trained inside the frozen model outperform their BC parents in the live game. The model matters not because BC requires it, but because it supplies an offline environment where PPO can explore alternatives without further interaction. At deployment, it instead provides temporal context for real-time control while generation is skipped. This split—generative during training, representational during acting—offers a route from offline gameplay to agents for environments that cannot wait for the policy. Whether it extends beyond binary-action levels, and how horizon and model uncertainty shape improvement, remain open.

Acknowledgments

We thank Clément Chatelain and Robin Condat for their guidance during the Representation Learning course at INSA Rouen Normandy, in which this project began, and CRIANN for providing the computing resources used for training. We thank Maël Planchot for contributing gameplay data. Geometry Dash is developed by RobTop Games.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 34, 2021.

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.

Maxime Burchi and Radu Timofte. TWISTER: Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.

Junliang Guo, Yang Ye, Tianyu He, Haoyu Wu, Yushu Jiang, Tim Pearce, and Jiang Bian. MineWorld: A real-time and open-source interactive world model on Minecraft. *arXiv preprint arXiv:2504.08388*, 2025.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.

Xianglong He, Chunli Peng, Zexiang Liu, Boyang Wang, Yifan Zhang, Qi Cui, Fei Kang, Biao Jiang, Mengyin An, Yangyang Ren, Baixin Xu, Hao-Xiang Guo, Kaixiong Gong, Size Wu, Wei Li, Xuchen Song, Yang Liu, Yangguang Li, and Yahui Zhou. Matrix-Game 2.0: An open-source real-time and streaming interactive world model. *arXiv preprint arXiv:2508.13009*, 2025.

Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. In *AAAI Conference on Artificial Intelligence*, 2018.

Anthony Hu, Václav Volhejn, Adrien Ramanana Rahary, Chris Mulder, Aditya Makkar, Alyx Liao, Amélie Royer, Manu Orsini, Adam Jelley, Eloi Alonso, Florian Laurent, Fredrik Norén, James Swingos, Jan Hünermann, Kent Rollins, Lucas Hosseini, Matthieu Le Cauchois, Maxim Peter, Pim de Witte, Tim Brown, Vincent Micheli, Moritz Böhle, Gabriel de Marmiesse, Viktoriia Sharmanska, Lucia Specia, Michael Black, and Patrick Pérez. Multiplayer interactive world models with representation autoencoders. *arXiv preprint arXiv:2607.05352*, 2026.

Yann LeCun. A path towards autonomous machine intelligence. *OpenReview*, 2022. Version 0.9.2.

Ted Li and Sean Rafferty. Playing Geometry Dash with convolutional neural networks. Stanford CS231N course report, 2017.

Bin Lin, Zongjian Li, Yuwei Niu, Kaixiong Gong, Yunyang Ge, Yunlong Lin, Mingzhe Zheng, JianWei Zhang, Miles Yang, Zhao Zhong, Liefeng Bo, and Li Yuan. iFSQ: Improving FSQ for image generation with 1 line of code. *arXiv preprint arXiv:2601.17124*, 2026.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, pages 2980–2988, 2017.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.

Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning*, 2024.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self supervised learning. *arXiv preprint arXiv:2505.12477*, 2025.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

Zaishuo Xia, Yukuan Lu, Xinyi Li, Yifan Xu, and Yubei Chen. Clone deterministic 3d worlds with geometrically-regularized world models. *arXiv preprint arXiv:2510.26782*, 2025.

Tongda Xu, Wendi Zheng, Jiajun He, José Miguel Hernández-Lobato, Yan Wang, Ya-Qin Zhang, and Jie Tang. Training-free vector quantization via gaussian VAEs. *arXiv preprint arXiv:2512.06609*, 2025.